

Zusammenfassung und Ausblick

Vom ladbaren Modul zur sicheren, beobachtbaren Kernelarbeit

2.10

Drei Entscheidungen prägen sicheren Kernelcode

Die passende Lösung ergibt sich aus Lebenszyklus, Ausführungskontext und gemeinsamem Zustand.

01**LEBENSZYKLUS**

Ressourcen vollständig anlegen, Fehlerpfade behandeln und beim Entladen synchron freigeben.

02**AUSFÜHRUNGSKONTEXT**

Vor jedem Aufruf klären: Darf dieser Pfad schlafen, blockieren oder auf Speicher warten?

03**NEBENLÄUFIGKEIT**

Gemeinsame Daten erkennen, Invarianten festlegen und den kleinstmöglichen Schutz wählen.

Beobachtbarkeit macht Annahmen mit Logs, Modulmetadaten, Tracepoints und Sanitizern überprüfbar.

Tag 3 bringt den Datenaustausch in den Fokus

Der nächste Schritt verbindet interne Kernelmechanismen mit belastbaren Schnittstellen zum User Space.

SPEICHER

Speicher sicher verwalten

Allokation, GFP-Flags, Lebensdauer
und sichere Freigabe im Kernel.

SCHNITTSTELLEN

User Space anbinden

Character Devices sowie
`copy_to_user()` und
`copy_from_user()`.

TREIBER

Gerätezugriffe strukturieren

`file_operations`, Datenpfade und
vertiefendes Debugging und Tracing.

Leitfrage: Wie wird aus internem Kernelcode eine robuste Schnittstelle zum User Space?